

VaayuDrishti

National Weather Big Data Analytics Platform

Full Workflow & Repository Structure Document

Problem ID	SIH26069 (26069)
Organization	Ministry of Earth Sciences (MoES)
Department	India Meteorological Department (IMD)
Category	Software
Theme	Disaster Management
Scope of this doc	Backend build (Phase 1) + Frontend specification (Phase 2)
Version	1.0
Date	30 September 2026

1. Problem Statement & Compliance Map

Design and develop a scalable National Weather Big Data Analytics Platform capable of collecting and processing real-time weather-related information for India from multiple internet-based sources including social media platforms, public datasets, websites, APIs, and citizen reports. The platform automatically collects weather-related posts tagged with #IMD and other relevant weather hashtags along with metadata (date & time, city, state, GPS location, photos, videos, event category) and stores them in a centralized database. Big data technologies and open-source tools support large-scale real-time ingestion, processing, storage and visualization. ML/AI techniques identify fake or misleading reports, verify untrusted sources, remove duplicates and automatically categorize weather events. A web-based dashboard and Admin Panel provide date-wise, event-wise and location-wise filtering, verification status tracking, real-time visualization and analytics.

Requirement traceability

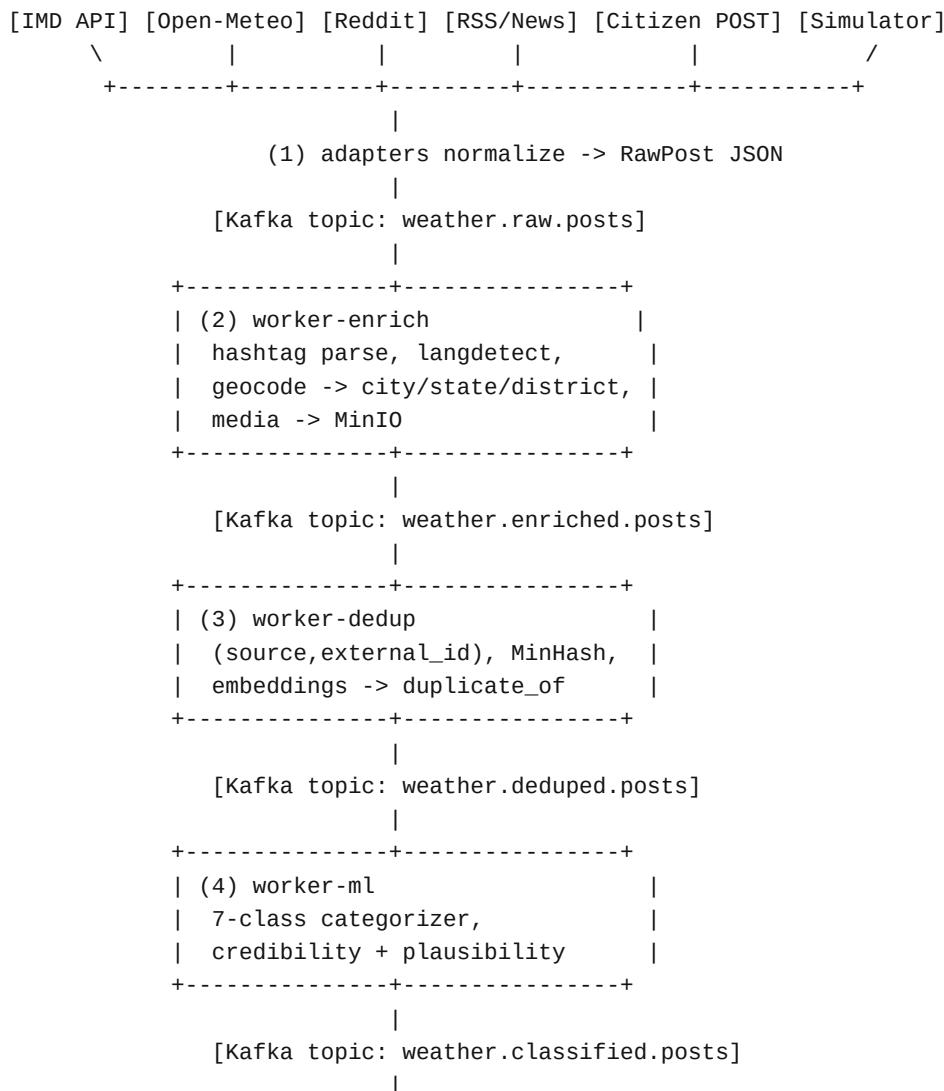
Multiple sources	6 adapters: social (Reddit/public JSON), IMD API, Open-Meteo API, RSS/news websites, public datasets, citizen POST /reports + high-volume simulator
#IMD + weather hashtags	Hashtag vocabulary (English + Hindi) matched in enrich stage; hashtags[] column + trending-hashtag analytics
Metadata: date/time	observed_at, ingested_at (timezone-aware IST)
Metadata: city/state	Offline Indian gazetteer + district GeoJSON point-in-polygon geocoding
Metadata: GPS	lat/lon + PostGIS geography(Point) column, bbox filter support
Metadata: photos/videos	Object storage (MinIO), media[] references, size/type validation
Metadata: event category	7 PS categories: rainfall, thunderstorms, flooding, heatwaves, fog, dust storms, strong winds (+ other)
Centralized database	PostgreSQL 17 + PostGIS as canonical store; OpenSearch replica for search/analytics
Big data / real-time	Redpanda (Kafka API) streaming backbone, 3-stage consumer pipeline, Spark structured streaming + batch rollups, MinIO, Redis
ML: fake/misleading	Credibility model (gradient boosting over engineered features incl. weather-plausibility cross-check vs IMD/Open-Meteo observations)
ML: verify untrusted	Source-reputation registry + human verification workflow (JWT admin panel APIs with audit trail)
ML: remove duplicates	MinHash LSH + multilingual sentence embeddings -> duplicate_of links
ML: auto-categorize	Zero-shot baseline -> fine-tuned DistilBERT 7-class classifier with confidence
Date/Event/Location filter	GET /api/v1/reports with combined filter parameters (built now, consumed by UI later)
Verification tracking	verification_status enum + admin verify/reject/bulk endpoints + audit_log
Real-time visualization	WebSocket live feed (Redis pub/sub) + /api/v1/geo/reports.geojson + analytics endpoints
Dashboard + Admin panel	Phase 2 frontend (specified in Section 7), API contract frozen in Phase 1

2. System Architecture

Docker Compose based, fully open-source, runnable on a single developer machine and horizontally scalable by increasing service replicas. Services:

postgres:17+PostGIS	Central database: reports, users, audit, rollups, reference observations
redpanda	Kafka-compatible streaming broker (topics, 4 partitions each)
opensearch:2	Full-text search + aggregation engine (report index)
minio	S3-compatible object storage for photos/videos
redis:7	Cache, rate limiting, pub/sub fan-out for the live WebSocket feed
api	FastAPI + uvicorn: REST API, JWT auth, WebSocket (port 8000)
ingestor	APScheduler running all source adapters -> raw.posts
worker-enrich	Geocoding, hashtag/lang parse, media download -> enriched.posts
worker-dedup	MinHash + embedding near-duplicate detection -> deduped.posts
worker-ml	Event classification + credibility scoring -> classified.posts, sink -> DB/index
spark-master/worker	Structured streaming + hourly/daily rollup jobs
seed	One-shot: migrations, admin user, reference geo data

End-to-end data flow



```
+-----+-----+
| (5) sink: idempotent upsert |
+-----+-----+
|           |           |
PostgreSQL  OpenSearch  Redis (live channel)
  ^
  | rollups (hourly/daily)
[Spark jobs]
  |
[FastAPI: REST + WebSocket] ---> future dashboard & admin panel
```

3. Step-by-Step Workflow

3.1 Ingestion (M1)

1. Scheduler (APScheduler) invokes each adapter on its own interval (IMD 10 min, Open-Meteo 15 min, Reddit 5 min, RSS 15 min, simulator continuous).
2. Adapter fetches source (httpx async), normalizes to RawPost schema: source, external_id, url, author, text, hashtags, lang, observed_at, lat/lon (if any), media urls, raw payload.
3. Producer publishes to weather.raw.posts (acks=all, idempotent). Adapter writes counters/errors to ingest_runs for the admin health endpoint.
4. Citizen reports: POST /api/v1/citizen/reports validated (rate limit, size, spam guard) -> stored pending -> injected into the same topic so every record follows one pipeline.

3.2 Processing pipeline (M2)

5. ENRICH: extract/normalize hashtags (#IMD, #MumbaiRains, Hindi variants); langdetect language; geocode text/GPS via offline gazetteer + district GeoJSON point-in-polygon -> city, state, district, geometry; download media to MinIO (type/size limits, SHA-256 key); emit enriched.posts.
6. DEDUP: exact match on (source, external_id); MinHash LSH over normalized text shingles; embedding cosine > 0.92 confirms near-duplicate -> set duplicate_of, skip sink for duplicates (counted for admin).
7. CLASSIFY: DistilBERT 7-class model returns event_category + confidence (zero-shot fallback when model artifact absent). Credibility model computes 0-1 score with feature contributions stored as JSONB.
8. PLAUSIBILITY: report location+time is checked against cached reference observations (IMD/Open-Meteo) - e.g. claimed '120mm rain' vs observed 0mm lowers credibility; disagreement raises a disputed flag.
9. SINK: idempotent upsert into reports (ON CONFLICT), index document into OpenSearch, publish to Redis channel weather:live for the WebSocket; all steps observable via pipeline metrics endpoint.

3.3 Storage & analytics (M5)

10. PostgreSQL holds the source of truth with GIST/btree/gin indexes; OpenSearch answers full-text + facet queries; MinIO holds media binaries referenced by reports.media[].
11. Spark jobs aggregate hourly/daily rollups (counts by category/state/source, avg credibility) into rollup_hourly / rollup_daily so analytics endpoints stay fast at scale.
12. Verification: admin reviewer inspects report + plausibility evidence via admin APIs, sets verification_status (verified/rejected/disputed) + note; audit_log records who/when/old/new.

4. Repository Folder Structure

Monorepo layout. Phase 1 builds backend/; frontend/ is specified here and built in Phase 2.

```
VaayuDrishti/
|-- docker-compose.yml          # all services, profiles: core/full/tools
|-- .env.example                # API keys (IMD), secrets, toggles
|-- Makefile                    # make up | down | migrate | seed | test | lint
|-- README.md                   # quickstart + demo script
|-- docs/
|   |-- VaayuDrishti_Full_Workflow.pdf # this document
|   |-- architecture.md
|   |-- api.md                  # endpoint contract for frontend team
|   |-- generate_pdf.py
|-- scripts/
|   |-- load_test.py            # 10k-post simulator blast + latency report
|   |-- demo.sh                 # 10-minute judge demo walkthrough
|-- data/
|   |-- reference/
|       |-- districts.geojson    # India districts (point-in-polygon)
|       |-- cities.csv          # offline gazetteer: city/state/district/lat/lon
|       |-- hashtags.txt        # weather hashtag vocabulary (en + hi)
|       |-- credibility_lexicon.txt
|-- backend/
|   |-- pyproject.toml          # deps + ruff/mypy/pytest config
|   |-- Dockerfile              # shared image for api/ingestor/workers
|   |-- alembic.ini
|   |-- migrations/
|   |   |-- env.py
|   |   |-- versions/          # 0001_initial, ...
|   |-- shared/
|   |   |-- config.py           # pydantic-settings (env-driven)
|   |   |-- db.py               # async engine/session, tx helpers
|   |   |-- models.py           # SQLAlchemy ORM (reports, ingest_runs, ...)
|   |   |-- schemas.py          # Pydantic RawPost/EnrichedPost/... contracts
|   |   |-- topics.py           # Kafka topic names + partitions
|   |   |-- kafka.py            # producer/consumer factory (aiokafka)
|   |   |-- redis_client.py      # cache + pub/sub helper
|   |   |-- objectstore.py       # MinIO client wrapper
|   |   |-- search.py           # OpenSearch index helper
|   |-- api/
|   |   |-- main.py             # app factory, CORS, routers, health
|   |   |-- deps.py             # DI: sessions, auth, pagination
|   |   |-- security.py         # JWT create/verify, password hashing, roles
|   |   |-- ws.py               # /api/v1/live/stream WebSocket endpoint
|   |   |-- routers/
|   |       |-- reports.py       # list/detail/filters/media
|   |       |-- analytics.py     # summary/timeseries/by-state/by-category/by-source/hashtags
|   |       |-- geo.py           # reports.geojson, states summary
|   |       |-- citizen.py       # POST reports, status tracking
|   |       |-- auth.py          # login, refresh, me
|   |       |-- admin.py         # verification, pipeline, ingest-runs, sources, audit, export
|   |-- ingest/
|   |   |-- base.py             # Adapter ABC + stats + error policy
|   |   |-- scheduler.py        # APScheduler entrypoint
|   |   |-- adapters/
|   |       |-- imd.py           # api.imd.gov.in + mausam fallback (key in .env)
|   |       |-- open_meteo.py    # keyless obs/forecast (also feeds plausibility)
|   |       |-- reddit.py        # public JSON endpoints, weather subreddits
|   |       |-- rss.py           # IMD bulletins + news RSS keyword filter
|   |       |-- citizen_db.py    # drains pending citizen reports -> topic
|   |       |-- simulator.py     # synthetic #IMD social stream (hi+en, true/false/dup)
```

```

| |-- pipeline/
| | |-- run_stage.py          # generic consumer loop (offsets, DLQ, metrics)
| | |-- enrich.py            # stage 2
| | |-- dedup.py             # stage 3
| | |-- classify.py           # stage 4 (category + credibility + plausibility)
| | |-- sink.py              # stage 5 (PG upsert, OS index, Redis live)
| |-- ml/
| | |-- geocode/             # gazetteer loader + point-in-polygon
| | |-- classify/             # model wrapper, label map, zero-shot fallback
| | |-- credibility/         # feature extraction + GBM model + plausibility
| | |-- dedup/               # minhash + embedding wrapper
| | |-- embeddings.py         # sentence-transformers loader (multilingual)
| | |-- train/
| | | |-- make_synthetic_labels.py
| | | |-- train_classifier.py
| | | |-- train_credibility.py
| | | |-- evaluate.py         # precision/recall/F1 report
| |-- spark/
| | |-- Dockerfile
| | |-- jobs/
| | | |-- stream_ingest.py    # structured streaming (raw -> PG batches)
| | | |-- rollups.py          # hourly/daily aggregation
| |-- tests/
| | |-- conftest.py          # testcontainers-style fixtures
| | |-- test_schemas.py
| | |-- test_geocode.py
| | |-- test_dedup.py
| | |-- test_classifier.py
| | |-- test_api_reports.py
| | |-- test_pipeline_e2e.py
|-- frontend/                # PHASE 2 (specified in Section 7, not built now)
| |-- package.json           # Vite, React 18, TypeScript, Tailwind
| |-- vite.config.ts
| |-- tailwind.config.js
| |-- src/
| | |-- main.tsx
| | |-- App.tsx              # router: public vs /admin guards
| | |-- api/client.ts         # fetch wrapper + typed endpoints
| | |-- stores/filters.ts     # global filter state (date/event/location/status)
| | |-- stores/auth.ts
| | |-- pages/
| | | |-- DashboardMap.tsx    # live map + KPI + charts
| | | |-- ReportsList.tsx     # table with all filters + pagination
| | | |-- ReportDetail.tsx    # shareable report page + media gallery
| | | |-- CitizenSubmit.tsx   # public report form + GPS + upload
| | | |-- TrackReport.tsx     # status by tracking token
| | | |-- admin/
| | | | |-- Login.tsx
| | | | |-- VerificationQueue.tsx
| | | | |-- PipelineHealth.tsx
| | | | |-- IngestRuns.tsx
| | | | |-- Sources.tsx
| | | | |-- AuditLog.tsx
| | |-- components/
| | | |-- map/IndiaMap.tsx    # MapLibre GL, clusters, layer toggles
| | | |-- map/TimeSlider.tsx
| | | |-- filters/FilterBar.tsx
| | | |-- charts/{TimeSeries,StateRank,SourceDonut,TrendHashtags}.tsx
| | | |-- kpi/KpiCards.tsx
| | | |-- report/ReportCard.tsx, MediaGallery.tsx, CredibilityBadge.tsx
| | | |-- ?-- layout/{Header,Sidebar,ThemeToggle}.tsx
|-- i18n/                    # en + hi dictionaries

```

5. Backend Specification (Phase 1 - Build Now)

5.1 Technology stack

Language	Python 3.11 (containers); typing + pydantic v2
API	FastAPI + uvicorn, async SQLAlchemy 2, Alembic migrations
Streaming	Redpanda 24.x (Kafka API), aiokafka consumers/producers
Database	PostgreSQL 17 + PostGIS 3.4
Search	OpenSearch 2.x
Objects	MinIO (S3 API)
Cache/bus	Redis 7
Batch	Apache Spark 3.5 (pyspark)
ML	scikit-learn, datasketch (MinHash), sentence-transformers (multilingual MiniLM), transformers (DistilBERT / bart-large-mnli zero-shot)
Quality	ruff, mypy, pytest (+ httpx TestClient, testcontainers)

5.2 Data model (PostgreSQL)

reports

```

id uuid PK | source text | external_id text | source_url text
author text | text text | lang text | hashtags text[]
media jsonb          # [{bucket,key,content_type,sha256}]
lat double precision | lon double precision | geom geography(Point,4326)
city text | district text | state text
event_category enum   # rainfall|thunderstorm|flooding|heatwave|fog|
                      # dust_storm|strong_winds|other
severity text | confidence numeric | observed_at timestamptz
ingested_at timestamptz | credibility_score numeric
credibility_features jsonb | plausibility jsonb
verification_status enum # pending|verified|rejected|disputed
verified_by uuid | verified_at timestamptz | verification_note text
duplicate_of uuid FK | is_duplicate bool | raw jsonb
UNIQUE(source, external_id) | GIST(geom), btree(event_category,
observed_at), gin(text gin_trgm)

```

Other tables

ingest_runs	id, source, started_at, finished_at, fetched, published, errors, last_error, status
admin_users	id, email, password_hash, full_name, role(admin reviewer viewer), is_active, created_at
audit_log	id, user_id, action, entity, entity_id, before jsonb, after jsonb, created_at
rollup_hourly	bucket_start, state, event_category, source, report_count, avg_credibility, verified_count
rollup_daily	same grain at day level
reference_observations	provider, station, lat, lon, observed_at, temp_c, precip_mm, wind_kph, payload jsonb
citizen_tokens	report_id, token, created_at (for public status tracking)

5.3 Kafka topics

weather.raw.posts	adapter output (4 partitions)
weather.enriched.posts	after geocode/media/lang
weather.deduped.posts	after duplicate marking
weather.classified.posts	after category + credibility
weather.verified.posts	successfully persisted records (audit/other consumers)
weather.dlq	poison messages with error metadata

5.4 Source adapters

imd	api.imd.gov.in city forecast/nowcast/warnings (free key via env IMD_API_KEY); graceful fallback when key absent
open_meteo	keyless hourly current weather for gazetteer cities; also populates reference_observations for plausibility
reddit	public JSON endpoints of weather/India subreddits, keyword+hashtag filter, polite UA, rate limit aware
rss	IMD bulletins + Indian news RSS (rainfall/flood/cyclone keywords) -> post normalization
citizen_db	drains citizen_reports flagged pending -> pipeline (keeps API write path decoupled)
simulator	generates realistic #IMD social posts: Indian cities, en/hi text, 7 categories, includes injected duplicates and fake claims for ML demo; rate configurable

5.5 ML components

Event classifier	7 PS categories; train: synthetic + rule-labeled corpus -> DistilBERT; inference fallback: bart-large-mnli zero-shot; output label + confidence
Credibility scorer	GBM/logistic over ~25 features: source trust tier, author age/followers (when available), sensationalism lexicon, exclamation/caps ratio, GPS-vs-text city mismatch, weather-plausibility delta, duplicate burst velocity, media presence, hashtag stuffing; output 0-1 + per-feature contributions (JSONB)
Plausibility engine	nearest reference observation (IMD/Open-Meteo) vs claimed phenomenon within +/- 3h and +/- 50km -> support ratio
Dedup	minhash LSH (64 perms, 5-shingle) + multilingual embeddings cosine >= 0.92; exact (source, external_id) guard first
Geocoder	offline gazetteer (~cities.csv) + district GeoJSON point-in-polygon; text city extraction with state hints; no external API dependency
Artifacts	backend/ml/artifacts/ (joblib/onnx) versioned + fallback chain: trained -> zero-shot -> keyword rules

5.6 REST + WebSocket API contract

GET	/health	liveness/readiness
GET	/api/v1/reports	filters: date_from, date_to, event[], state, district, city, verification_status, source, q, bbox, min_credibility, page, page_size, sort
GET	/api/v1/reports/{id}	full detail + credibility features
GET	/api/v1/reports/{id}/media/{key}	signed-ish media streaming

```

POST  /api/v1/reports                citizen submit (multipart, rate-limit)
GET    /api/v1/reports/track/{token} public verification status
GET    /api/v1/analytics/summary      KPIs (totals, verified %, live rate)
GET    /api/v1/analytics/timeseries   bucket=hour|day, group_by category/state
GET    /api/v1/analytics/by-state      counts + avg credibility per state
GET    /api/v1/analytics/by-category   counts + trend delta
GET    /api/v1/analytics/by-source     platform distribution
GET    /api/v1/analytics/hashtags     trending hashtags
GET    /api/v1/geo/reports.geojson    map layer (bbox + filters)
WS     /api/v1/live/stream            new report events (redis pub/sub)
--- auth: POST /api/v1/auth/login -> JWT(access,refresh); Bearer required ---
GET    /api/v1/auth/me
PATCH /api/v1/admin/reports/{id}/verification verify|reject|dispute+note
POST   /api/v1/admin/reports/bulk-verification bulk actions
GET    /api/v1/admin/pipeline         kafka lag, rates, stage health
GET    /api/v1/admin/ingest-runs       adapter health table
PATCH /api/v1/admin/sources/{name}    enable/disable adapter
GET    /api/v1/admin/audit             audit trail
GET    /api/v1/admin/export?format=csv filtered dataset export

```

5.7 Security & non-functional

- JWT access (30 min) + refresh (7 d); roles admin > reviewer > viewer enforced by dependency.
- Argon2/bcrypt password hashing; citizen endpoints rate-limited per IP (Redis token bucket).
- Idempotent consumers, DLQ for poison messages, graceful shutdown (SIGTERM drains).
- httpx timeouts + retries with backoff; every adapter isolated so one failing source never blocks others.
- Structured JSON logs + /api/v1/admin/pipeline exposes consumer lag and throughput for the demo.
- Media hard limits (10 MB/image, 50 MB/video, mime allowlist), EXIF GPS optionally harvested.

6. Build Milestones (backend)

M0 - Scaffold & infrastructure

- docker-compose.yml with all services + profiles; Dockerfile; Makefile; .env.example
- shared/: config, db, models, schemas, kafka/redis/minio/opensearch helpers
- Alembic initial migration; seed job (admin user + reference geo data); /health green

M1 - Ingestion

- Adapter base + scheduler + ingest_runs accounting
- IMD, Open-Meteo, Reddit, RSS, citizen, simulator adapters publishing to weather.raw.posts

M2 - Pipeline

- enrich (geocode/lang/media/hashtags) -> dedup (minhash+embeddings) -> classify (category+credibility) -> sink (PG+OS+Redis)
- Idempotency, DLQ, per-stage counters; end-to-end record visible with city/state/category/dup flag

M3 - API

- Reports list/detail with all filters; analytics endpoints; geo.json; media streaming
- Citizen submit + tracking; JWT auth; admin verification + pipeline + ingest + audit + export; WebSocket live feed

M4 - ML

- Synthetic label generation, classifier training/eval report, credibility model training
- Plausibility engine wired to reference_observations; artifacts + fallback chain

M5 - Spark, hardening & demo

- Spark hourly/daily rollups into rollup_* tables; analytics endpoints read rollups
- Load test (10k posts), latency report, pytest suite green, ruff+mypy clean
- README + scripts/demo.sh: 10-minute judge walkthrough

7. Frontend Specification (Phase 2 - Not Built Now)

The backend exposes the complete contract above; the frontend is a pure consumer and can be built independently afterwards.

7.1 Stack

Framework	Vite + React 18 + TypeScript
Styling	Tailwind CSS, dark/light theme
Map	MapLibre GL (India boundaries, clusters, hexbin heatmap, time slider)
Charts	ECharts (time series, state rank, source donut, hashtags)
Data	TanStack Query (REST) + native WebSocket for live feed
State	Zustand global filter store shared by map/charts/table
Routing	React Router; public routes vs /admin guarded by JWT role
i18n	English + Hindi dictionaries

7.2 Public dashboard pages

- DashboardMap: live India map (clustered markers colored by category), KPI cards, stacked time-series, state ranking, source donut, trending hashtags, verified-alerts ticker; layer toggles (category/status/source); time-slider replay of last 24h/7d.
- FilterBar (global): date range presets, event multi-select, state->district->city cascade, verification status, source; drives every widget.
- ReportsList: paginated table with all filters, sort, credibility score column, quick-verify badges.
- ReportDetail: shareable URL, full metadata, media gallery, credibility breakdown, verification timeline, nearby reports.
- CitizenSubmit: text + category + city/GPS 'use my location' + photo/video upload + CAPTCHA; success returns tracking token.
- TrackReport: verification status by token.

7.3 Admin panel pages (JWT)

- Login; VerificationQueue: pending reports side-by-side with plausibility evidence, one-click Verify/Reject/Flag, bulk actions, reviewer notes.
- PipelineHealth: Kafka lag per topic, ingestion rate/min, adapter last-success (WebSocket-refreshed).
- IngestRuns, Sources (enable/disable), AuditLog, CSV/JSON export of filtered datasets.

7.4 Real-time & UX

- WebSocket feeds new reports to map + charts without refresh; optimistic UI for admin actions.
- Responsive desktop-first, skeletons/empty states/error toasts, accessible color-coded severity.

8. Demo & Verification Script (judge walkthrough)

1. docker compose up -d --profile full -> make migrate seed -> health endpoints green.
2. Show live Kafka topic counters rising (simulator + Open-Meteo + Reddit adapters).
3. Blast: python scripts/load_test.py --count 10000 -> show pipeline throughput + end-to-end latency report.
4. Query one record through its lifecycle: raw -> enriched (city/state/GPS) -> dedup (duplicate flagged) -> classified (category+credibility) -> sink.
5. Fake-detection: show a fabricated '120mm rain' report vs reference observation -> low plausibility -> disputed suggestion; vs a corroborated report -> high score.
6. Admin APIs via /docs: verify/reject a report, audit log entry appears.
7. Analytics: date/event/location filters combined; timeseries + state ranking; Spark rollup speeds the query.
8. WebSocket: new event appears in live feed instantly; GeoJSON layer ready for the map.
9. pytest / ruff / mypy run green (quality gate).

Success criteria: all PS requirements traceable (Section 1), pipeline observable end-to-end, ML components demonstrable with metrics, API contract frozen for the frontend team.